Complete Explanation of register.php

This file handles user registration - it receives form data from `index.html`, validates everything, saves files, and stores user information.

1. PHP Session Start (Line 2)

```
<?php
session_start();
```

- **What it does:** Starts a PHP session
- **Why:** Allows the script to store information in `$_SESSION` that persists across pages
- **Use here:** Will store registration data so the user can log in later

2. Get Form Data from index.html (Lines 4-10)

```
$profilepic = $_FILES['profilepic'];
$fname      = $_POST['fname'];
$lname      = $_POST['lname'];
$email      = $_POST['email'];
$password   = $_POST['pass'];
$course     = $_POST['course'];
$doc        = $_FILES['doc'];
```

Breaking it down:

- **`$_FILES['profilepic']`** — Gets the uploaded profile picture file
  - `$_FILES` is for file uploads (not `$_POST`)
  - Contains an array with: `['name']`, `['tmp_name']`, `['error']`, `['size']`, etc.

- **`$_POST['fname']` through `$_POST['course']`** — Gets text input values
  - `'fname'` = First name
  - `'lname'` = Last name
  - `'email'` = Email address
  - `'pass'` = Password (note: it's called `'pass'` in the form)
  - `'course'` = Selected course/department
- **`$_FILES['doc']`** — Gets the uploaded document file

Key point: These come from the form in `index.html` with matching `name` attributes

3. Validate All Required Fields (Lines 13-19)

```
// basic input check: all text fields and both files must be provided
if (
  empty($fname) || empty($lname) || empty($email) || empty($password)
|| empty($course)
) {
  echo "Please fill in all the required fields.";
  exit();
}
```

What it does:

- **`empty()`** — Checks if a variable is empty (no value, zero, or false)
- **`||`** — OR operator (if ANY field is empty, the condition is true)
- **If any field is empty:**
  - Displays an error message
  - `exit()` — Stops the script (prevents further processing)

In simple terms: Make sure the user filled in first name, last name, email, password, AND selected a course. If anything is missing, stop and show error.

4. Check for Profile Picture Upload Errors (Lines 22-26)

```
// verify no upload errors occurred
if ($profilepic['error'] !== 0) {
  echo "Error uploading profile picture.";
  exit();
}
```

What it does:

- **`$profilepic['error']`** — A code that tells if the upload had errors
  - `0` = No error (successful)
  - Any other number = Error occurred
- **`!== 0`** — "NOT equal to 0" (means there WAS an error)
- **If error detected:** Show error message and stop

In simple terms: Did the profile picture upload successfully? If not, show error and stop.

5. Check for Document Upload Errors (Lines 27-31)

```
if ($doc['error'] !== 0) {
  echo "Error uploading document.";
  exit();
}
```

Same logic as above, but for the document file

6. Validate Profile Picture File Type (Lines 34-40)

```
// validate the extensions of the uploaded files
$allowedImages = ['jpg', 'jpeg', 'png'];
$profileExt = strtolower(pathinfo($profilepic['name'],
PATHINFO_EXTENSION));
```

```
if (!in_array($profileExt, $allowedImages)) {
  echo "Only JPG/PNG allowed for profile picture.";
  exit();
}
```

Breaking it down:

- **`$allowedImages = ['jpg', 'jpeg', 'png']`** — An array of allowed file types
  - Array is like a list: `[item1, item2, item3]`
- **`pathinfo($profilepic['name'], PATHINFO_EXTENSION)`** — Extracts the file extension
  - `$profilepic['name']` = The filename (e.g., "photo.jpg")
  - `PATHINFO_EXTENSION` = Gets just the extension part ("jpg")
- **`strtolower()`** — Converts to lowercase
  - If user uploads "Photo.JPG", becomes "jpg" (for consistent comparison)
- **`in_array($profileExt, $allowedImages)`** — Checks if the extension is in the allowed list
  - Returns `true` if found, `false` if not
  - `!` before it means "NOT found"
- **If file type NOT allowed:** Show error and stop

In simple terms: Only allow JPG and PNG images. If the user uploads a BMP or GIF, reject it.

7. Validate Document File Type (Lines 42-48)

```
$allowedDocs = ['pdf', 'doc', 'docx'];
$docExt = strtolower(pathinfo($doc['name'], PATHINFO_EXTENSION));
if (!in_array($docExt, $allowedDocs)) {
  echo "Only PDF/DOC/DOCX allowed for document.";
  exit();
}
```

Same logic as profile picture, but allows PDF, DOC, and DOCX

8. Move Profile Picture to Folder (Lines 51-56)

```
// move the files into place
$profilepicname = time() . "_" . basename($profilepic['name']);
$profilepicpath = "profiles/" . $profilepicname;
if (!move_uploaded_file($profilepic['tmp_name'], $profilepicpath)) {
  echo "Failed to save profile picture.";
  exit();
}
```

Breaking it down:

- **`time()`** — Gets current Unix timestamp (e.g., 1234567890)
  - Used to make filename unique (prevents overwriting if two users upload "photo.jpg")
- **`basename($profilepic['name'])`** — Gets just the filename (without folders)
  - e.g., "photo.jpg" from "/Users/john/photo.jpg"
- **`$profilepicname = time() . "_" . basename($profilepic['name'])`** — Creates unique filename
  - Example: "1234567890_photo.jpg"
- **`$profilepicpath = "profiles/" . $profilepicname`** — Full path
  - Example: "profiles/1234567890_photo.jpg"
- **`move_uploaded_file($profilepic['tmp_name'], $profilepicpath)`** — Moves file
  - `$profilepic['tmp_name']` = Temporary location where PHP stored the uploaded file
  - `$profilepicpath` = Final location in "profiles" folder
  - Returns `true` if successful, `false` if failed
  - `!` means "if NOT successful"
- **If move failed:** Show error and stop

In simple terms: Take the uploaded file, give it a unique name with timestamp, move it to the profiles folder.

9. Move Document to Folder (Lines 58-63)

```
$docname = time() . "_" . basename($doc['name']);
$docpath = "docs/" . $docname;
if (!move_uploaded_file($doc['tmp_name'], $docpath)) {
  echo "Failed to save document.";
  exit();
}
```

Same logic as profile picture, but saves to "docs" folder

10. Set Cookies (Lines 66-68)

```
// set cookies for convenience
setcookie("student_email", $email, time() + 86400 * 30, "/");
setcookie("student_name", "$fname $lname", time() + 86400 * 30, "/");
```

What it does:

- **`setcookie(name, value, expiration, path)`** — Creates a cookie (stored on user's browser)
  - **name:** Cookie identifier ("student_email", "student_name")
  - **value:** What to store ($email, full name)
  - **expiration:** When cookie expires (time() + 86400 * 30 = 30 days from now)
    - `86400` = seconds in 1 day
    - `* 30` = 30 days
  - **path:** "/" = available on entire website

Why cookies? For convenience - browser remembers user info

11. Store Session Data (Lines 71-77)

```
// store consistent session data for dashboard/login (including
password)
$_SESSION['email']    = $email;
$_SESSION['password'] = $password;          // plaintext for demo only
$_SESSION['name']     = "$fname $lname";
$_SESSION['profile']  = $profilepicpath;
$_SESSION['course']   = $course;
```

What it does:

- Stores user information in PHP session (server-side, more secure than cookies)
- **`$_SESSION['email']`** — So login.php can check if email matches
- **`$_SESSION['password']`** — So login.php can check if password matches
- **`$_SESSION['name']`** — Full name (concatenated with space)
- **`$_SESSION['profile']`** — Path to profile picture (so dashboard can display it)
- **`$_SESSION['course']`** — Course code (cs, math, stats)

⚠️ **Security Note:** Comment says "plaintext for demo only" - in real apps, passwords should be hashed!

12. Convert Course Code to Full Name (Lines 80-93)

```
// compute a human-readable course name for later use
switch ($course) {
  case 'cs':
    $courseName = "Computer Science";
    break;
  case 'math':
    $courseName = "Mathematics";
    break;
  case 'stats':
    $courseName = "Statistics";
    break;
  default:
```

```
    $courseName = "Unknown Department";
    break;
}
```

What it does:

- **`switch ($course)`** — Checks the value of `$course` variable
  - `$course` contains "cs", "math", or "stats" (from the form)
- **`case 'cs':`** — If `$course` is "cs"
  - `$courseName = "Computer Science"` — Set readable name
  - `break;` — Stop checking other cases
- **`default:`** — If none of the cases match
  - Uses "Unknown Department" as fallback

In simple terms: Convert short course codes to full names. If user selected "cs", change it to "Computer Science".

13. Store Course Name in Session (Lines 96-97)

```
// (optional) keep the readable name in session too
$_SESSION['courseName'] = $courseName;
```

Stores the human-readable course name in session

- So dashboard can display "Computer Science" instead of "cs"

14. Redirect to Login (Lines 100-101)

```
// everything succeeded, send user to login page
header('Location: login.php');
exit();
```

What it does:

- **`header('Location: login.php')`** — Redirects the browser to login.php
    - User automatically goes to login page after successful registration
- **`exit()`** — Stops script execution

In simple terms: If everything worked, send the user to the login page so they can log in with their new account.

Complete Flow Diagram

```
User submits form from index.html
         ↓
PHP receives: name, email, password, course, profile pic, doc
         ↓
Check if all fields filled → NO → Show error, STOP
         ↓
                    YES ↓
Check if files uploaded without errors → NO → Show error, STOP
         ↓
                    YES ↓
Check if profile pic is JPG/PNG → NO → Show error, STOP
         ↓
                    YES ↓
Check if document is PDF/DOC/DOCX → NO → Show error, STOP
         ↓
                    YES ↓
Move profile pic to "profiles/" folder → FAILED → Show error, STOP
         ↓
                    SUCCESS ↓
Move document to "docs/" folder → FAILED → Show error, STOP
         ↓
                    SUCCESS ↓
Set cookies (student_email, student_name)
         ↓
Store in $_SESSION (email, password, name, profile path, course)
         ↓
Convert course code to full name (cs → Computer Science)
         ↓
```

```
Store course name in $_SESSION
         ↓
Redirect to login.php ✓
         ↓
User can now log in!
```

Key Concepts Summary

Concept	Purpose
`$_POST`	Get text input values from forms
`$_FILES`	Get uploaded files
`empty()`	Check if value is blank
`pathinfo()`	Extract file extension
`strtolower()`	Convert text to lowercase
`in_array()`	Check if value exists in a list
`move_uploaded_file()`	Move file from temp to permanent location
`setcookie()`	Store data on user's browser (30 days)
`$_SESSION`	Store data on server (until logout)
`switch/case`	Check one variable against multiple values
`header()`	Redirect to another page

That's the complete breakdown! You can now explain to others how registration works, what each part does, and why certain security checks are needed. Let me know if you have any questions!